ALGORITHMS & COMPLEXITY
LECTURE 3

We introduce now formal tools to measure (with some sort of relaxation on constant multiplication) the rate of growth of a_n once all indices $n \geq n_0$ (i.e. for sufficiently large n)

Recall that a sequence $\{a_n\}_{n\geq 0}$ can be viewed as a function

$$f: \mathbb{N} \longrightarrow \mathbb{R} \quad \text{s.t.} \quad f(n) = a_n$$
$$\mathbb{N} = \{0, 1, 2, \ldots\}$$

Def. 1.

Let $f, g : \mathbb{N} \to \mathbb{R}$. We say that g dominates f (or f is dominated by g) if there exists constant $M \in \mathbb{R}_+$ & index $k_0 \in \mathbb{N}$ such that

$$\boxed{|f(n)| \leq M \cdot |g(n)|} \quad \forall n \geq k_0$$

$$\left[\text{or} \quad |a_n| \leq M|b_n| \quad \forall n \geq k_0 \right]$$

If both $f(n) \geq 0$ & $g(n) \geq 0$ (i.e. $f(n) = a_n$, $g(n) = b_n$) then we have a shorter notation $\boxed{0 \leq a_n \leq M b_n}$

ALGORITHMS & COMPLEXITY
LECTURE 3

In computer science usually a_n represents positive values as it counts (time, memory consumption, number of basic calculations etc).

Geometrically Def 1 means: (for simplicity assume $f, g \geq 0$)
as we consider the values $f(1)=a_1$, $f(2)=a_2$, $f(3)=a_3$, ... and $g(1)=b_1$, $g(2)=b_2$, $g(3)=b_3$, ... there is a point (namely k_0) after which the size of $f(n)$ is bounded above by the positive multiple M of size of $g(n)$

We say if Def 1 holds

$$a_n = O(b_n)$$

$$[\, f(n) = O(g(n)) \,]$$

" a_n is of order b_n "

Example: $a_n = 5n^2 + 3n + 1$

$b_n = n^2$

Obviously $|a_n| \leq 5n^2 + 3n^2 + n^2 \quad \forall n \geq \overset{k_0}{\overset{\|}{1}}$

$|a_n| \leq 9n^2$

ALGORITHMS & COMPLEXITY
LECTURE 3

$\Rightarrow \quad a_n = O(n^2) \Rightarrow \boxed{O(b_n) = a_n}$

Note however that we also have

$$|b_n| = n^2 \leq 5n^2 + 3n + 1$$

so in fact $\boxed{b_n = O(a_n)}$

In such a case we say $a_n = \Theta(b_n)$ "a_n is of SHARP ORDER of b_n". Here a_n is dominated by b_n & b_n is dominated by a_n.

Def. 2: We say $a_n = \Theta(b_n)$ (of sharp order) if $a_n = O(b_n)$ & $b_n = O(a_n)$.
The latter means $\exists M_1 \exists k_0 \forall n \geq k_0$

$$|a_n| \leq M_1 |b_n|$$

& $\exists M_2 \exists k_0 \forall n \geq k_0$

$$|b_n| \leq M_2 |a_n|$$

For the case when both a_n & $b_n \geq 0$ it suffices to show the existence of $M_1, M_2 > 0$ & k_0, $\forall n \geq k_0$

$\boxed{0 \leq M_1 b_n \leq a_n \leq M_2 b_n} \quad \Leftrightarrow a_n = \Theta(b_n)$ & $b_n = \Theta(a_n)$

ALGORITHMS & COMPLEXITY
LECTURE 3

Example: $a_n = a_t n^t + a_{t-1} n^{t-1} + \dots + a_2 n^2 + a_1 n + a_0$

$t \in \mathbb{N}$, $a_t \neq 0$, $a_t, a_{t-1}, \dots, a_0 \in \mathbb{R}$

since $|x+y| \leq |x| + |y|$

$$|a_n| = |a_t n^t + a_{t-1} n^{t-1} + \dots + a_2 n^2 + a_1 n + a_0|$$

$$\leq |a_t n^t| + |a_{t-1} n^{t-1}| + \dots + |a_2 n^2| + |a_1 n| + |a_0|$$

$$= n^t |a_t| + n^{t-1} |a_{t-1}| + \dots + n^2 |a_2| + n |a_1| + |a_0|$$

$$\leq n^t |a_t| + n^t |a_{t-1}| + \dots + n^t |a_2| + n^t |a_1| + n^t |a_0|$$

$$= n^t \underbrace{\left(|a_t| + |a_{t-1}| + \dots + |a_2| + |a_1| + |a_0| \right)}_{M}$$

$$= M n^t \quad \Rightarrow \quad |a_n| \leq M n^t \quad \forall n \geq \underbrace{1}_{k_0}$$

$$\Rightarrow \boxed{a_n = O(n^t)}$$

on the other hands $|a_n| \geq n^t |a_t|$

$$\Rightarrow \boxed{a_n = \Omega(n^t)}$$

In particular if a_n does not explode at all $|a_n| \leq M \cdot 1$ bounded sequence

$$\Downarrow$$

$$\boxed{a_n = O(1)}$$

In computer science we have typical rates of growth for a_n:

ALGORITHMS & COMPLEXITY
LECTURE 3

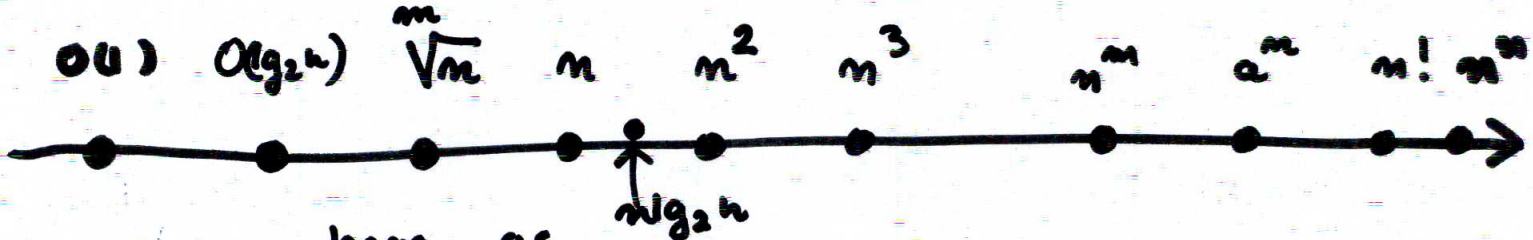
If a_n is of order:

- $O(1)$ – constant
- $O(\lg_2 n)$ – logarithmic
- $O(\sqrt{n})$ – square-root
- $O(n)$ – linear order
- $O(n \lg_2 n)$ – $n \lg_2 n$
- $O(n^2)$ – quadratic
- $O(n^3)$ – cubic
- $O(n^4)$ – quartic
- $O(n^m)$ – polynomial $m = 0, 1, 2, 3, \ldots$
- $O(n^\alpha)$ – $\alpha \in \mathbb{R}$ $\alpha \geq 1$ power α
- $O(a^n)$ – $a > 1$ exponential
- $O(n!)$ – factorial

in computer sc.

These are the most common orders, but one can put them on the axis from left to right to symbolize which one is slower and which one is faster

ALGORITHMS & COMPLEXITY
LECTURE 3



here as

$1 \le \lg_2 n \le n \qquad n \ge 2$

$n \le n\lg_2 n \le n^2$

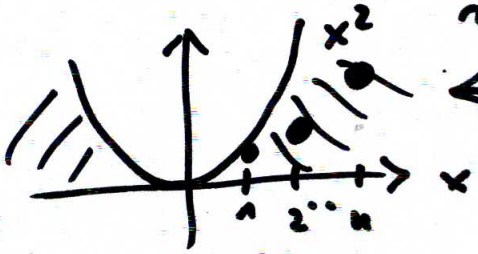
One can squeeze ∞ - many other orders on the axis:

e.g. between n^2 & n^3 $\quad n^2\lg_2 n$

e.g between n & n^2 $\quad n^\alpha \rightarrow n^{\frac{3}{2}}, n^{2.9}$

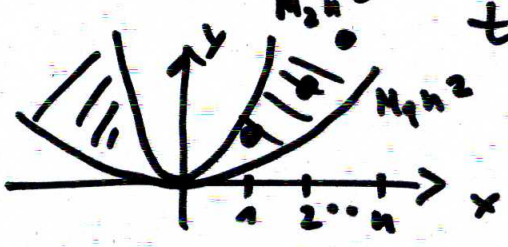
for any $1 < \alpha < 2$

To see the explosion growth it is good to see at the graphs of functions say $a_n = O(n^2)$ It means a_n is below n^2 (but still can be below n)



← It can be here

If $a_n = \Theta(n^2)$ then it is between two parabolas



ALGORITHMS & COMPLEXITY
LECTURE 3

Some useful properties (not difficult to be proved)

$$a_n = O(b_n) \Rightarrow c \cdot a_n = O(b_n)$$

$$\left.\begin{array}{l} a_n = O(b_n) \\ c_n = O(b_n) \end{array}\right\} \Rightarrow a_n \pm c_n = O(b_n)$$

$$\left.\begin{array}{l} a_n = O(b_n) \\ c_n = O(d_n) \end{array}\right\} \Rightarrow a_n \cdot c_n = O(b_n \cdot d_n)$$

$$\left.\begin{array}{l} a_n = O(b_n) \\ b_n = O(c_n) \end{array}\right\} \Rightarrow a_n = O(c_n)$$

$$\left.\begin{array}{l} a_n = O(c_n) \\ b_n = O(d_n) \end{array}\right\} \Rightarrow a_n \pm b_n = O(\max\{c_n, d_n\})$$

Note that: $a_n = O(b_n)$ only means that a_n is bounded from above by b_n

but a_n can still be of lower order

$$a_n = O(c_n)$$

but a_n is of order $a_n = O(d_n)$

ALGORITHMS & COMPLEXITY
LECTURE 3

if $\boxed{a_n = O(b_n) \Rightarrow a_n = O(d_n)}$!

$\forall c_n$ s.t. that for $b_n \leq d_n$

if $a_n = O(b_n)$ $\Rightarrow$ (i) a_n may be of order $O(c_n)$ $c_n \leq b_n$

or

(ii) a_n may not be of order $O(c_n)$ $c_n \leq b_n$

all depends whether $a_n = \Theta(b_n)$

if yes then (i) is false (ii) true

if no then (i) is true (ii) false.

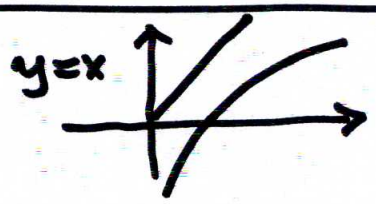
Example: Let us analyze an example as $\ln(x \cdot y) = \ln x + \ln y$

$$a_n = \ln(n!) = \ln[1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n]$$

$$a_n = \ln 1 + \ln 2 + \ln 3 + \ldots + \ln(n-1) + \ln n$$

(i)

Naive estimate:



$\ln x$ $x \in [0, n]$ $\ln x \leq n$

a)

b) $\ln x \leq x$

$$0 \leq a_n \overset{a)}{=} \ln 1 + \ldots + \ln n \leq \underbrace{n + n + \ldots + n}_{n\text{-times}} = n^2$$

$\Rightarrow \boxed{a_n = O(n^2)}$ (i) can we improve it?

ALGORITHMS & COMPLEXITY

LECTURE 3

(ii) b) more intricate approach

$$0 \le a_n = \ln 1 + \ln 2 + \ln 3 + \ldots + \ln n \le 1 + 2 + \ldots + n$$

$$= \frac{(1+n)(n)}{2} = \frac{1}{2}(n^2+n) \le \frac{1}{2}(n^2+n^2) = n^2$$

$$\Rightarrow \boxed{a_n = O(n^2)} \quad (\bullet\bullet)$$

So a more intricate approach does not improve previous result (•)

(iii) More advanced approach:

This does not mean that $a_n = \Theta(n^2)$ (i.e. we cannot improve this estimate).

Usually we plot now (n, a_n) and compare with plotted graphs x^2, $x^{3/2}$, x, $x\lg_2 x$, $\lg_2 x$ (in Matlab or in Mathematica)

What we will notice that points (n, a_n) are well below (n, Kn^2). In fact it is close to $x\lg_2 x$.

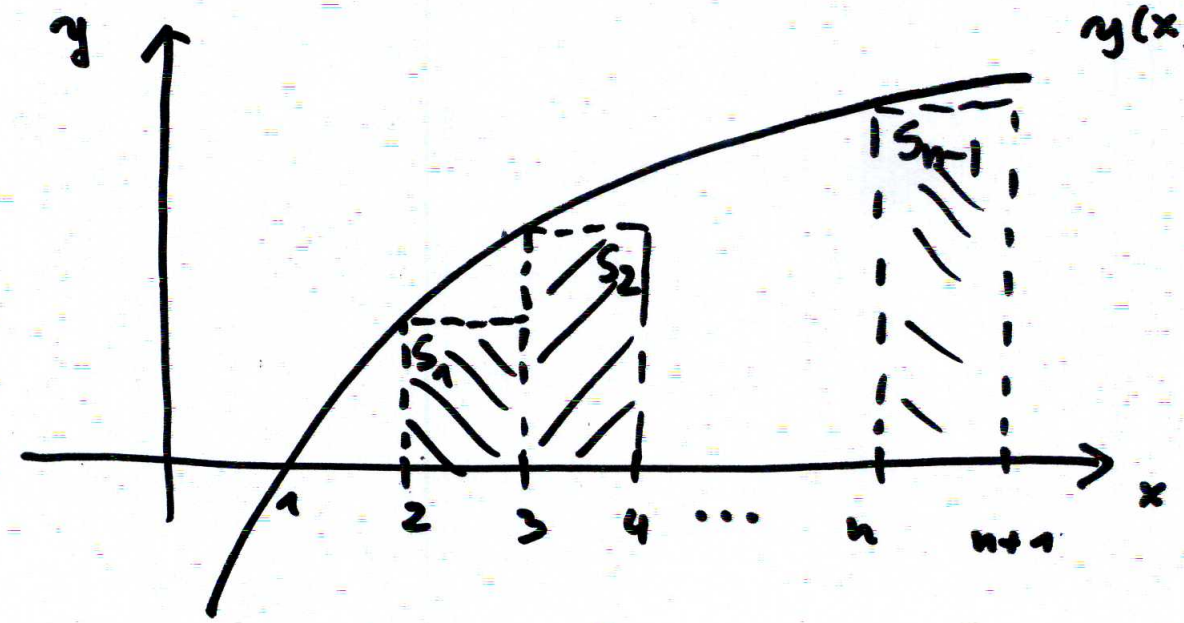
We shall prove that now by resorting to the integration. We show

(▲) $a_n \le M_2 n\lg_2 n \quad \forall n \ge k_1$ $\qquad$ $a_n \ge (\blacktriangle)\, M_1 n\lg_2 n \quad \forall n \ge k_2$

$$\Downarrow$$

$$\boxed{a_n = \Theta(n\lg_2 n)}$$

ALGORITHMS & COMPLEXITY
LECTURE 3



Areas of $S_1, S_2, \dots, S_{n-1}$ are smaller than area $\int_1^{n+1} \ln x\,dx$ bounded by OX axis $(1 \le x \le n+1)$ & the graph of $y(x) = \ln x$

Let A_i = area of S_i. We have

$$A_1 + A_2 + \dots + A_{n-1} \le \int_1^{n+1} \ln x\,dx$$

$$(3-2)\ln 2 + (4-3)\ln 3 + \dots + [(n+1)-n]\ln n \le \int_1^{n+1} \ln x\,dx$$

$$\underbrace{\ln 1}_{\text{as } \ln 1 = 0} + \ln 2 + \ln 3 + \dots + \ln n \le \int_1^{n+1} \ln x\,dx$$

$$a_n \underset{(*)}{\le} \int_1^{n+1} \ln x\,dx$$

{ we use integration by parts to compute right-hand side of (*) } — I.P.

recall that $\int_a^b f'(t)g(t)\,dt = f(t)g(t)\Big|_a^b - \int_a^b f(t)g'(t)\,dt$

I.P. formula

ALGORITHMS & COMPLEXITY

LECTURE 3

$$\int_1^{n+1} \ln x\,dx = \int_1^{n+1} x' \ln x\,dx \overset{I.P.}{=} x\ln x\Big|_1^{n+1} - \int_1^{n+1} x(\ln x)'\,dx$$

$$= \Big((n+1)\ln(n+1) - \underbrace{1\cdot\ln 1}_{=0}\Big) - \int_1^{n+1} 1\,dx$$

$$= n\ln(n+1) + \ln(n+1) - x\Big|_1^{n+1}$$

$$= n\ln(n+1) + \ln(n+1) - (n+1-1)$$

$$= n\ln(n+1) + \ln(n+1) - n$$

$$\Rightarrow 0 \le a_n \le n\ln(n+1) + \ln(n+1) - n$$

$$\le n\ln(n+1) + n\ln(n+1)$$

$$= 2n\ln(n+1)$$

but $\ln(x+1) \le 2\ln x$

$e^{\ln(x+1)} \le e^{2\ln x}$

$x+1 \le x^2$

$x^2 - x - 1 = 0$

$\Delta = 1+4 = 5$

$x_{1,2} = \frac{1 \pm \sqrt{5}}{2}$

we take $x_2 = \frac{1}{2} + \frac{\sqrt{5}}{2}$

$$\le 2n \cdot 2\ln n$$

$$= 4n\ln n \quad \underline{\forall n \ge K_2}$$

So for $x \ge \frac{1}{2} + \frac{\sqrt{5}}{2}$

$x^2 \ge x+1 \Leftrightarrow \ln(x+1) \le 2\ln x$

K_2 is any integer $> \frac{1}{2} + \frac{\sqrt{5}}{2}$

$$\ln n = \lg_e n = \lg_e 2 \cdot \lg_2 n$$

$$\Downarrow$$

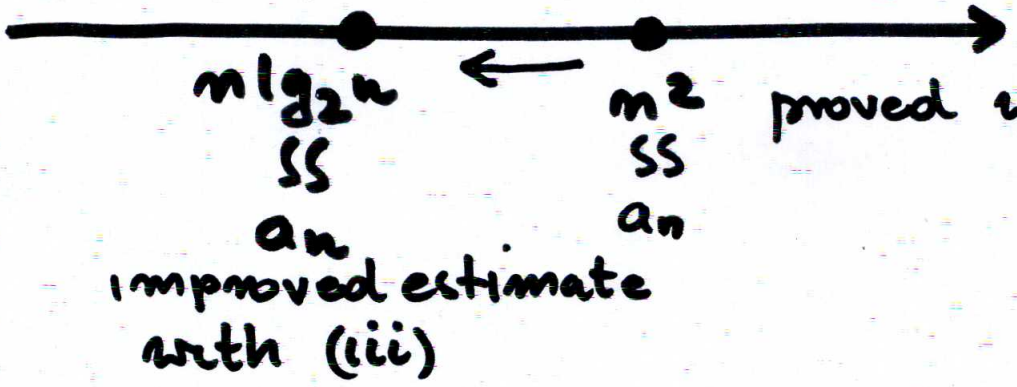
$$0 \le a_n \le \underbrace{4 \cdot \lg_e 2}_{M_2} n \lg_2 n$$

$$\Downarrow$$

$$\boxed{a_n = O(n \lg_2 n)} \quad (1)$$

So we have now a better estimate instead of $a_n = (n^2)$

ALGORITHMS & COMPLEXITY
LECTURE 3

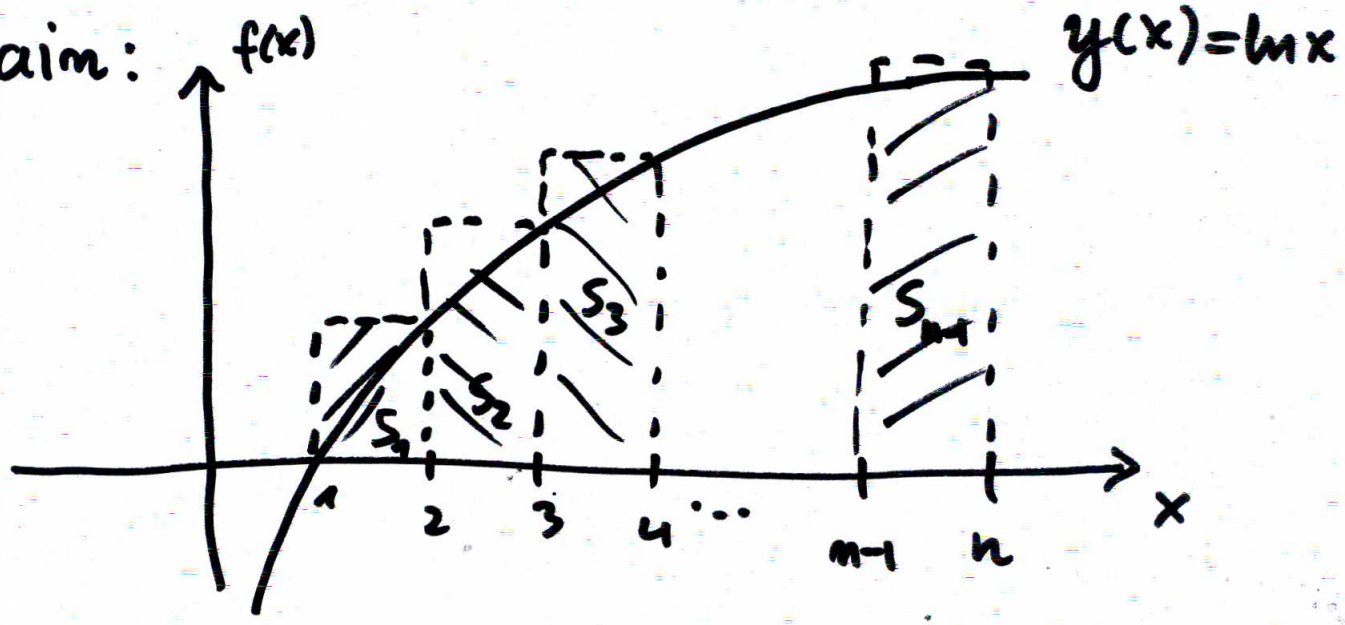


Question: can we improve (iii) more? (i.e. can we shift estimate to left?)

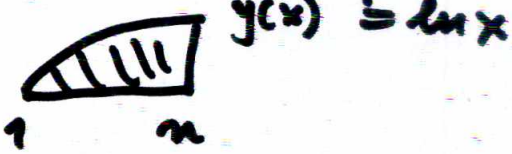
We will show now that no!!!

i.e. $\exists M_1 \; \exists K_1 \quad M_1 \, n \lg_2 n \leq a_n \quad \forall n \geq K_1$

Again:



Areas of $S_1, S_2, \ldots, S_{n-1}$ summed up exceed the area of



ALGORITHMS & COMPLEXITY
LECTURE 3

$$A(S_1)+A(S_2)+\dots+A(S_{n-1}) \geq \int_1^n \ln x\,dx$$

$$(2-1)\ln 2+(3-2)\ln 3+\dots+[n-(n-1)]\ln n \geq \int_1^n \ln x\,dx$$

$$\underbrace{\ln 1}_{=0}+\ln 2+\ln 3+\dots+\ln n \geq x\ln x\Big|_1^n - \int_1^n x(\ln x)'\,dx \quad \left((\ln x)'=\tfrac{1}{x}\right)$$

$$a_n$$

$$a_n \geq n\ln n - \underbrace{\ln 1}_{=0} - x\Big|_1^n = n\ln n - n + 1$$

$$a_n \geq n\ln n - n + 1 \geq n\ln n - n$$

$$a_n \geq n(\ln n - 1) \qquad \forall n \in \mathbb{N} \quad (\blacksquare)$$

but $\ln(x)-1 \geq \frac{1}{2}\ln x$ for $x \geq x_0$

$$e^{\ln x - 1} \geq (e^{\ln x})^{\frac{1}{2}}$$

$$\frac{x}{e} \geq \sqrt{x}$$

$$\frac{x^2}{e^2} \geq x \Leftrightarrow$$

$$x^2 - e^2 x > 0$$

$$x(x-e^2) > 0$$

$$x \geq e^2$$

we have $(\blacksquare)$ take $k_1 \geq e^2$

$$a_n \geq n(\ln n - 1) \geq n\,\tfrac{1}{2}\ln n \quad \text{for } k_2 \leq n \text{ (fixed)}$$

$$a_n \geq \underbrace{\tfrac{1}{2}\lg e^2}_{M_1}\, n\lg_2 n \qquad \forall n \geq k_1$$

$$\Downarrow$$

$$\boxed{a_n \geq M_1 n\lg_2 n} \quad (2)$$

$$\Rightarrow \text{by (1) \& (2)} \quad \boxed{a_n = \Theta(n\lg_2 n)}$$

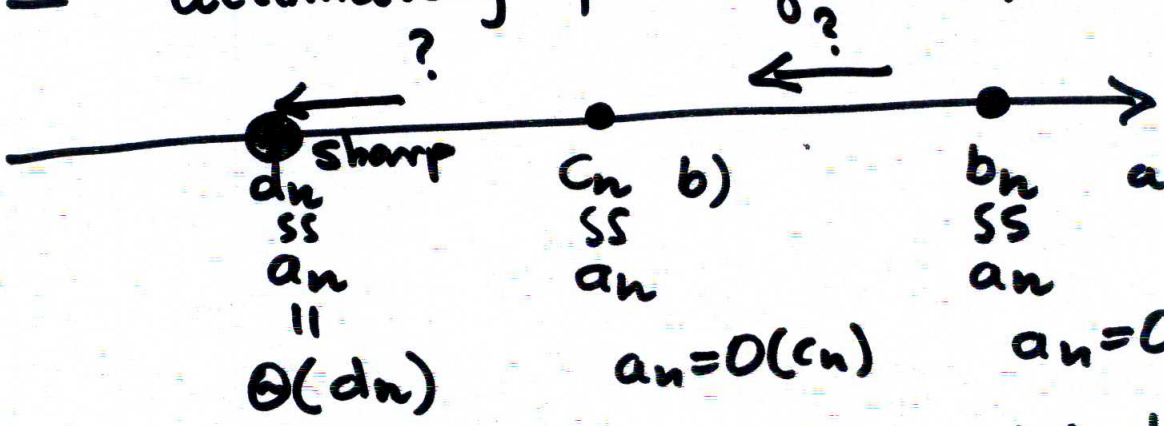
LECTURE 3
ALGORITHMS & COMPLEXITY

The example from above shows the general / usual path when estimating the asymptotics for a_n coming from the specific algorithm

a) — finding "terse" estimate

b) — searching for improvements

c) — ultimately proving sharpness



Usually more & more advanced techniques are needed in passing from a) $\Rightarrow$ c)
One can in a search for improvements use computer plots for (n, a_n) against plotted different $(n, f(n))$.

There are no standard techniques for all algorithms.
At best if we can calculate exactly a_n (see. e.g. Bubble sort)

this gives sharpness in one step without passing through a) & b).

However finding exact formula is sometimes difficult. And also we are interested when we compare 2 algorithms for performing a certain task how they differ for sufficiently large n (modulo constant). For the latter

$a_n = O(f(n))$ & $\hat{a}_n = O(g(n))$

↑ for Algorithm 1 ↑ for Algorithm 2

these measurements suffice !!!

Criterion of using integration is nice but can be applied only if the sequence a_n is the sum (which usually is the case in computer science – see e.g. Bubble sort).

However we may have situations in the analysis of algorithms that this technique cannot be applied. Then the analysis each time involves the adapted to the problem new argument.

ALGORITHMS & COMPLEXITY
LECTURE 3

REMARK:

a) Quite often the analysis of algorithms involves counting all possibilities (search space)

e.g. if we want to find a max. element the question is about complexity (of list)
of the exhausitve search Algorithm we simply take all $f(x_i)$

b) Other case is when we want to assess the worst-scenario so that we can compare performance of the algorithms against worst case.

All is related to the analysis & complexity of algorithms and resorts to either:

- exact computation of all possibilities (exact sums, combinatorial counting)

or

- approximate computation of all possibilities inequalities, counting (they all involve $a_n = a_1 + \dots$